EMPLOYEE ATTENDANCE DASHBOARD

Viva Questions and Answers

Questions a teacher or examiner can ask, with simple answers

Project: Employee Attendance Dashboard (AttendTrack)

Made by: Cecil Baraik, B.Sc. IT

Contains: project, architecture, Java, Spring Boot, database, security, React, testing, Docker

Contents

How to use this guide . . . 3
A. Project basics . . . 3
B. Architecture and design . . . 4
C. Java and Spring Boot . . . 5
D. Database . . . 7
E. Security . . . 7
F. Fake check-in prevention . . . 9
G. Business rules . . . 9
H. Frontend (React) . . . 10
I. Reports and alerts . . . 11
J. Testing . . . 11
K. Deployment and tools . . . 12
L. Limitations, future scope and tricky questions . . . 13
Quick revision - one-line answers . . . 14

How to use this guide

- Answers are short and simple, so you can say them in your own words.
- Start with the **first 10 questions** - teachers almost always ask these.
- If you do not know an answer, say what you know and connect it to your project. Never invent.
- Keep the project running on your laptop during the viva. Showing a feature is better than explaining it.

> **Tip:** Best demo order (3 minutes): login as admin > Dashboard > light/dark switch > Attendance check-in > login as employee > apply leave > login as manager > approve > Reports > Export PDF > Kiosk QR.

A. Project basics

Q1. Tell me about your project.

My project is an **Employee Attendance Dashboard** called AttendTrack. It is a full-stack web application. Employees check in and out, apply for leave and request corrections. Managers approve requests and see their team. Admins see the whole company, manage users, holidays and shifts, and download reports. It also stops fake check-ins using a rotating QR code, GPS and office network rules. The backend is **Spring Boot (Java)**, the frontend is **React**, the database is **H2 / MySQL**, and it runs with **Docker**. It has light and dark mode.

Q2. Why did you choose this project?

Many small offices still use paper registers or Excel for attendance. It is slow, people can sign for friends, and monthly calculation takes time. I wanted to solve a real problem and also learn real industry tools like Spring Boot, React, security with JWT, and Docker.

Q3. What is the objective of the project?

To record attendance correctly and quickly, stop fake attendance, make leave approval easy, and give managers live reports instead of manual calculation.

Q4. Who are the users of your system?

Three roles: **Admin** (HR / owner - full access), **Manager** (only their department) and **Employee** (only their own data).

Q5. What are the main modules?

Login and security, Dashboard, Attendance, Employees, Self-service (My Attendance and My Requests), Approvals, Check-in security (QR, GPS, IP), Holidays and shifts, Reports and export, Notifications (email alerts), Audit log, and Users.

Q6. What makes your project different from a normal attendance project?

Most student projects only have add/view attendance. Mine also has:

- real security: JWT, BCrypt, roles, account lock, audit log
- fake check-in prevention: rotating QR code, GPS geofence, office IP check
- shifts including night shifts, holidays and overtime
- Excel and PDF reports, scheduled email alerts
- 47 automated tests, Docker and MySQL production setup
- light and dark mode

Q7. Is this a real software or only a college project?

It is built like real software: proper login security, database, tests, and Docker for deployment. A small company could use it. But for real big companies, it would also need things like biometric devices, payroll and a mobile app.

Q8. What type of application is it?

A **web application**, specifically a **Single Page Application (SPA)** frontend with a **REST API** backend. It follows the **client-server** model.

Q9. How long did it take and how did you build it?

I built it feature by feature: first the basic dashboard, then security, self-service, fake check-in prevention, rules (holidays, shifts, overtime), reports and alerts, and finally the production setup. After each feature I ran the tests and pushed the code to GitHub.

Q10. What did you learn from this project?

Building REST APIs with Spring Boot, database design with JPA, authentication with JWT and Spring Security, React components and state, connecting frontend and backend, writing tests, and deploying with Docker. I also learned to think about security problems like brute force and fake check-ins.

B. Architecture and design

Q11. Explain the architecture of your project.

It is a **3-tier architecture**: the React frontend (presentation), the Spring Boot backend (business logic) and the database (data). The frontend sends HTTP requests with JSON to the backend. The backend checks the JWT token, applies the rules, and reads or writes the database using JPA/Hibernate.

Q12. What layers does your backend have?

Controller (receives requests), **Service** (business logic), **Repository** (database access) and **Model/Entity** (tables). Plus security, config, DTO and exception packages.

Q13. Why did you separate controller, service and repository?

Separation of concerns. Each layer has one job, so code is cleaner, easier to test and easier to change. For example I changed H2 to MySQL without changing the service or controller code.

Q14. What is the flow when an employee checks in?

React sends POST /api/me/check-in with the token. The JWT filter verifies the token. Spring Security checks the role. CheckInPolicyService checks QR, GPS and IP rules. AttendanceService checks the shift and decides Present or Late, then saves the record through the repository. Hibernate runs the SQL INSERT. The backend returns JSON and React updates the screen.

Q15. Is your project MVC?

The backend uses a layered architecture that is similar to MVC: the controller is the C, the entities/DTOs are the M, and the View is not in Spring - it is the separate React frontend. So it is MVC split across a REST API and a SPA.

Q16. Why did you keep frontend and backend separate?

They can be developed, tested and deployed independently. The same API can later be used by a mobile app. React gives a fast, smooth user interface without page reloads.

Q17. Draw / explain your database design.

Main tables: Employee, AttendanceRecord, UserAccount, LeaveRequest, CorrectionRequest, Holiday, Shift, CheckInSettings, Notification and AuditLog. One employee has many attendance records and many leave requests (one-to-many). A user account is linked to one employee (one-to-one). Many employees share one shift (many-to-one).

Q18. Why is UserAccount separate from Employee?

An employee is a person in the company; a user account is a login. Not every employee needs a login, and the admin account may not be an employee. Keeping them separate also keeps password data away from normal employee data.

Q19. How do you make sure an employee has only one attendance record per day?

There is a unique rule on (employee, date) and the service checks it before saving. If someone tries to check in twice, the API returns an error (409 Conflict).

C. Java and Spring Boot

Q20. What is Spring Boot? Why did you use it?

Spring Boot is a Java framework that makes it fast to build web applications. It has an embedded Tomcat server, auto-configuration and starter dependencies, so I do not need to set up a server manually. It is the industry standard for Java backends.

Q21. What is the difference between Spring and Spring Boot?

Spring is the main framework (dependency injection, MVC, etc.) but needs a lot of configuration. Spring Boot is built on Spring and adds auto-configuration, an embedded server and starters, so the app runs with very little setup.

Q22. What is Dependency Injection?

Instead of a class creating its own objects with "new", Spring creates them (beans) and gives them through the constructor. Example: AttendanceService receives AttendanceRecordRepository. It makes code loosely coupled and easy to test.

Q23. What is a Bean?

An object that Spring creates and manages. Classes marked with @Service, @Repository, @RestController, @Component or methods with @Bean become beans.

Q24. Explain the annotations you used.

- **@SpringBootApplication** - main class, starts everything
- **@RestController** - class that handles API requests and returns JSON
- **@GetMapping / @PostMapping / @PutMapping / @DeleteMapping** - map URL + method
- **@Service / @Repository** - business logic / database classes
- **@Entity, @Id, @ManyToOne** - map a class to a table
- **@PreAuthorize** - role check before a method runs
- **@Transactional** - all database steps succeed or none
- **@Scheduled** - run a method at fixed times
- **@Valid, @NotBlank, @Email** - check input

Q25. What is application.properties?

The settings file of Spring Boot: server port (8080), database URL, office start time, grace minutes, JWT expiry, mail settings and so on. We also have application-prod.properties for production.

Q26. What are Spring profiles?

Different settings for different environments. Our default (dev) profile uses H2 and demo data. The "prod" profile uses MySQL, takes passwords from environment variables, and hides error details.

Q27. What is Maven and pom.xml?

Maven is the build tool for Java. pom.xml lists the project information and dependencies (Spring Boot, JWT, POI...). Maven downloads them, compiles the code, runs tests (mvn test) and makes the jar file.

Q28. What is @Transactional?

It makes a method run as one database transaction. If any step fails, all changes are rolled back. Example: when a correction is approved, the request status and the attendance record are updated together.

Q29. How do you handle errors in the backend?

A GlobalExceptionHandler (@RestControllerAdvice) catches exceptions and returns a clean JSON message with the right status code (400, 403, 404, 409). Stack traces are never shown to the user.

Q30. What is a record in Java?

A short way to write a data class (Java 16+). It automatically has a constructor, getters, equals and toString. I used records for DTOs and for the settings classes.

Q31. Which Java version did you use?

The project targets Java 17 (an LTS version). I run it on JDK 21, which is also LTS.

D. Database

Q32. Which database did you use and why?

H2 for development - no installation, it saves to a file and starts with the app. **MySQL 8** for production - a reliable database server used by most hosting companies.

Q33. What is JPA and Hibernate? What is ORM?

ORM (Object Relational Mapping) maps Java classes to database tables. JPA is the Java standard for ORM, and Hibernate is the implementation that actually generates SQL. So I save a Java object and Hibernate writes the INSERT query.

Q34. How did you create the tables?

From the entity classes. Hibernate creates or updates the tables automatically (spring.jpa.hibernate.ddl-auto=update).

Q35. What is a Repository in Spring Data?

An interface that extends JpaRepository. Spring creates the code for save, findAll, findById, delete. I can also write method names like findByEmployeeCodeIgnoreCase and Spring creates the query.

Q36. How do you switch from H2 to MySQL?

Run with the prod profile and give the MySQL URL, username and password as environment variables (DB_URL, DB_USERNAME, DB_PASSWORD). Or simply run docker compose, which starts MySQL too. No Java code changes are needed.

Q37. What is a primary key and foreign key in your project?

Every table has an id as primary key. AttendanceRecord has employee_id as a foreign key pointing to Employee.

Q38. What is normalization? Is your database normalized?

Normalization means removing duplicate data by splitting it into related tables. Yes - for example employee details are stored once in Employee, and attendance records only store the employee id.

Q39. How is SQL injection prevented?

JPA uses parameterised queries, so user input is never joined directly into SQL. Input is also validated.

E. Security

Q40. How does login work in your project?

The user sends username and password to /api/auth/login. The backend checks the password against the BCrypt hash. If correct, it creates a JWT token and sends it back. React stores it and sends it in the Authorization header with every request. The JWT filter checks it each time.

Q41. What is JWT? What are its parts?

JSON Web Token - a signed token. It has 3 parts separated by dots: **header** (algorithm), **payload** (username, role, expiry) and **signature**. I sign it with HMAC-SHA256 and a secret key, so it cannot be changed.

Q42. Why JWT and not sessions?

JWT is stateless: the server does not store sessions, so it is simpler to scale and works well with a separate React frontend and a future mobile app.

Q43. What if someone steals a JWT token?

They could use it until it expires (8 hours), so tokens are short-lived. If a password is changed or reset, the token version changes and old tokens stop working. In production HTTPS is used so tokens cannot be read on the network.

Q44. How are passwords stored?

As BCrypt hashes, never as plain text. BCrypt adds a random salt and is slow, so even if the database is stolen, the passwords are very hard to crack.

Q45. What is the difference between hashing and encryption?

Encryption can be reversed with a key. Hashing is one-way - you cannot get the password back. For passwords we use hashing; to log in we hash the typed password and compare.

Q46. What is the difference between authentication and authorization?

Authentication = who are you (login). Authorization = what you can do (role and department checks).

Q47. How do you stop brute-force attacks?

After 5 wrong passwords the account is locked for 15 minutes. For QR codes, 5 wrong codes block the person for 10 minutes. All failed logins go into the audit log.

Q48. How does a manager see only their own department?

The AccessGuard class checks the manager's linked employee department on every request. Even if someone changes the URL or calls the API directly, the backend refuses. The frontend hiding a button is not enough - the backend enforces it.

Q49. What is CORS and why did you configure it?

Cross-Origin Resource Sharing. Browsers block calls to an API from another origin unless allowed. The frontend runs on port 5173 and the backend on 8080, so I allowed only the frontend origin.

Q50. Why did you disable CSRF?

CSRF attacks use cookies that the browser sends automatically. My API does not use cookies - the token is sent in a header by our JavaScript - so CSRF is not possible and the protection is not needed.

Q51. What is the audit log for?

Accountability. It records who did what, when and from which IP: logins, failed logins, data changes, approvals and rejected check-ins. Admins can check it if something goes wrong.

Q52. What other security measures did you add?

Password policy, account lockout, token version, no self-approval, X-Forwarded-For trusted only from known proxies, security headers in nginx (CSP, X-Frame-Options), plain-text emails, no stack traces in production, and the app refuses to start in production without a strong JWT secret.

F. Fake check-in prevention

Q53. What is buddy punching and how does your project stop it?

Buddy punching is when a friend marks attendance for someone who is not present. I stop it with three rules the admin can combine: a **rotating QR code** on an office screen, a **GPS geofence**, and an **office network (IP)** check.

Q54. How does the rotating QR code work?

Like Google Authenticator (TOTP). The server has a secret key. Every 30 seconds it calculates HMAC-SHA256(secret, time window) and turns it into a 6-digit code. The kiosk shows it as a QR code. The employee scans it, and the server calculates the code again and compares. A photo of the code becomes useless after about 30-60 seconds.

Q55. Why do you accept the previous code also?

If someone scans at second 29 and the request reaches the server at second 31, the code has already changed. Accepting one previous window avoids false failures.

Q56. What is constant-time comparison?

Comparing two codes in the same time whether they match early or not. A normal comparison stops at the first wrong digit, and an attacker could measure the time to guess digits. Constant-time stops that timing attack.

Q57. How does the GPS check work?

The browser gives the phone's latitude and longitude. The backend uses the **Haversine formula** to calculate the distance to the office. If it is more than the allowed radius (for example 200 m), check-in is refused.

Q58. Can GPS be faked?

Yes, on rooted phones with fake-GPS apps. That is why GPS is combined with the QR code and IP rules. Using two or more rules together makes cheating much harder.

Q59. How does the office network rule work?

The admin adds the office public IP or a CIDR range like 192.168.1.0/24. The backend checks if the request IP is in the list. The X-Forwarded-For header is trusted only when it comes from our own proxy, so it cannot be faked from outside.

Q60. Why do phones need HTTPS for QR scan and GPS?

Browsers allow the camera and location only on secure (https) pages, for privacy. That is why I added "npm run dev:phone", which starts the frontend with a local HTTPS certificate.

G. Business rules

Q61. How do you decide if an employee is late?

If the check-in time is after the shift start plus the grace time, the status is Late. Default: 09:30 + 15 minutes, so after 09:45 is Late. If the employee has a shift, its own start and grace are used.

Q62. What is a half day?

If the employee worked less than 4.5 hours (check-out minus check-in), the day is marked Half day.

Q63. How do you calculate overtime?

Overtime = hours worked minus the standard hours of the shift (default 8). On weekends and holidays every hour is overtime.

Q64. How do you handle night shifts?

A night shift like 22:00-06:00 crosses midnight. If the check-out time is earlier than the check-in time, we add 24 hours, so 22:00 to 06:00 = 8 hours.

Q65. How are holidays handled?

Holidays are not working days. Nobody is marked absent, leave over a holiday does not use the balance, and reports skip them. The admin can add national holidays with one click and festivals manually.

Q66. How does leave balance work?

Each year: Casual 12, Sick 10, Earned 15, and Unpaid without limit. Only working days are counted. Pending requests are also counted so nobody can apply for more than they have.

Q67. How is the attendance percentage calculated?

Days attended (present, late, half day) divided by (working days minus leave days), times 100.

Q68. How does the correction request work?

If an employee forgot to check out, they send the correct times with a reason. The manager approves, and the attendance record is updated automatically, with an audit log entry.

H. Frontend (React)

Q69. What is React? Why did you use it?

React is a JavaScript library for building user interfaces with reusable components. The screen updates automatically when data changes. It is fast, popular and easy to maintain.

Q70. What is a component?

A reusable piece of the UI written as a function that returns JSX. Example: StatusBadge shows a coloured status label and is used on many pages.

Q71. What are props and state?

Props are inputs passed from a parent component (read-only). **State** is data inside a component that can change, and when it changes React re-renders the component.

Q72. Which React hooks did you use?

useState (store values like form data), **useEffect** (load data from the API when the page opens), **useContext** (share login and theme across the app), and useCallback / useMemo for performance.

Q73. What is JSX?

A syntax that lets us write HTML-like code inside JavaScript. It is converted into normal JavaScript during the build.

Q74. What is Vite?

A fast build tool and dev server. It starts instantly, reloads on save, and builds optimised files for production. Its proxy sends /api calls to the backend on port 8080 during development.

Q75. What is a Single Page Application?

The browser loads one HTML page once; after that React changes the content without full page reloads. React Router changes the URL and shows the right page.

Q76. How did you make light and dark mode?

All colours are CSS variables. Dark mode changes the data-theme attribute on the html tag, and all variables switch together. The choice is saved in localStorage. On the first visit it follows the system setting using prefers-color-scheme. Charts also use theme colours.

Q77. How does the frontend talk to the backend?

Through one file, api.js, using fetch. It adds the JWT token to the header, converts JSON, and if the server returns 401 it logs the user out.

Q78. How do you show different menus for different roles?

Each menu item has a list of allowed roles. App.jsx shows only items allowed for the logged-in user's role. But the real protection is in the backend.

I. Reports and alerts

Q79. How do you create Excel and PDF files?

Apache POI for Excel (.xlsx) and **OpenPDF** for PDF. The backend builds the file in memory and sends it as a download. CSV is plain text with commas.

Q80. How do the scheduled email alerts work?

Spring's @Scheduled with cron expressions. "0 30 10 * * MON-FRI" sends the missing check-in reminder at 10:30 on weekdays; the weekly summary goes on Monday at 09:00. Emails are sent with Spring Mail over SMTP.

Q81. What happens if email is not configured?

The email is not sent but saved in the Notifications table with status "Logged", so the feature can still be shown and tested. With SMTP settings, the status becomes "Sent".

Q82. What is a cron expression?

A text that defines a schedule. In Spring it has 6 parts: second, minute, hour, day, month, weekday. Example: 0 0 9 * * MON = 9:00 every Monday.

Q83. What is "Needs attention" in reports?

It lists people who were late or absent 3 or more times in the selected period, with the pattern, for example "late 7 times (mostly Mondays)".

J. Testing

Q84. How did you test your project?

47 automated tests with **JUnit 5**, **Spring Boot Test** and **MockMvc**. They test attendance rules, login and security, leave and corrections, check-in rules, shifts and holidays, reports and alerts, and production settings. I run them with "mvn test". I also tested manually in the browser in both themes.

Q85. What is MockMvc?

A Spring testing tool that sends fake HTTP requests to the controllers without starting a real server. I used it to check, for example, that an employee gets 403 when calling an admin API.

Q86. What is unit testing vs integration testing?

A unit test checks one small part alone (like the late calculation). An integration test checks many parts together (controller + security + database). My project has both.

Q87. Give an example of a test case.

Check-in at 09:50 when the shift starts at 09:30 with 15 minutes grace should give status LATE. Another: a manager trying to approve their own leave should get 403 Forbidden. In the code these are the tests `onTimeCheckInIsPresentAndLateIsLate` and `managerCannotApproveOwnOrOtherDepartmentRequests`.

K. Deployment and tools

Q88. What is Docker? Why did you use it?

Docker packages the app with everything it needs into a container, so it runs the same on any computer or server. One command starts the database, backend and frontend.

Q89. Difference between an image and a container?

An image is the package (like a class). A container is a running copy of that image (like an object).

Q90. What is Docker Compose?

A tool to run many containers together from one YAML file. My docker-compose.yml starts MySQL, the backend and the nginx frontend in the right order.

Q91. What is a multi-stage Docker build?

The first stage uses a big image with Maven/Node to build the app; the final stage copies only the built jar or files into a small image. The final image is smaller and safer.

Q92. What is nginx doing in your project?

It serves the built React files, forwards /api requests to the backend, and adds security headers. It also hides the actuator and H2 console from outside.

Q93. What is the .env file?

It holds secrets like database passwords and the JWT secret for Docker. It is in .gitignore so it is never pushed to GitHub.

Q94. What is Git? Which commands did you use?

Git is a version control system. I used git status, git add, git commit -m "message", git push and git log. The code is on GitHub in my Web-Development repository.

Q95. How would you deploy it on the internet?

Rent a Linux server (VPS), install Docker, clone the repo, fill the .env file and run docker compose up. Then add a domain and HTTPS using Caddy or certbot. I wrote all steps in DEPLOYMENT.md.

Q96. What is the health check endpoint?

/actuator/health from Spring Boot Actuator returns {"status":"UP"} when the app is working. Docker uses it to know if the backend is healthy.

L. Limitations, future scope and tricky questions

Q97. What are the limitations of your project?

No fingerprint or face recognition, no payroll, no native mobile app, only one company, and GPS can be faked on rooted phones (so it is combined with QR and IP).

Q98. What is the future scope?

Face recognition or biometric device integration, payroll module, mobile app with notifications, two-factor login for admins, multi-company cloud version, and CI/CD with GitHub Actions.

Q99. What if two people share one password?

The friend still needs to be in the office to see the QR code and be on the office network. Fully stopping it needs biometrics, which is in my future scope. All check-ins record IP and method in the audit trail.

Q100. What if the server time is wrong?

Check-in time comes from the server, not from the user's phone, so users cannot change it. The server timezone is set to Asia/Kolkata. Servers sync time automatically with NTP.

Q101. What happens if the internet goes off in the office?

This version needs a connection to the server. A manager can later mark attendance on the Attendance page, or the employee can send a correction request.

Q102. How many users can your system handle?

For a small or medium company (hundreds of employees) it works well. The backend is stateless, so for more users we can run more backend copies behind nginx and use MySQL with proper indexes.

Q103. Did you write all the code yourself?

Answer honestly. For example: "I built it step by step with help from AI tools and documentation, but I understand every part, I tested it, and I can explain and change any feature." Then be ready to show code.

Q104. Show me where the late status is calculated.

In the backend, AttendanceService (method statusForCheckIn), which uses WorkCalendar to get the employee's shift start and grace minutes.

Q105. Where is the JWT created?

In security/JwtService.java (method generate). It is checked in JwtAuthFilter for every request.

Q106. Where are the roles checked?

In SecurityConfig (all /api routes need login), @PreAuthorize on controller methods, and AccessGuard for department checks.

Quick revision - one-line answers

Term	One line
Spring Boot	Java framework with embedded server and auto-configuration
REST API	Data through URLs and HTTP methods (GET, POST, PUT, DELETE) in JSON
JPA / Hibernate	Maps Java classes to database tables (ORM)
JWT	Signed token: header.payload.signature; stateless login
BCrypt	Slow, salted one-way password hash
Spring Security	Authentication + authorization framework
RBAC	Permissions by role: Admin, Manager, Employee
React	Component-based JavaScript UI library
Vite	Fast dev server and build tool
SPA	One page, content changes without reload
TOTP / HMAC	Time-based code from a secret key (rotating QR)
Haversine	Distance between two GPS points
CIDR	IP range, e.g. 192.168.1.0/24
CORS	Browser rule for calling another origin
Docker	App + everything it needs in a container
nginx	Web server and reverse proxy
Maven	Java build and dependency tool (pom.xml)
JUnit / MockMvc	Automated tests / fake HTTP requests
H2 vs MySQL	Development file DB vs production DB server
Audit log	Record of who did what, when, from where